

RAPORT PROJEKTU – Moodify

Jakub Liszewski

Kamil Niezabitowski

Eryk Majcherczak

Filip Kotyński

Wiktor Musiał

Dawid Krawczyk

Kamil Labus

Michał Kuźnicki

Szymon Stalica

Spis treści

Streszczenie wykonawcze.....	5
Opis projektu	5
Cel główny	5
Rozpoznawane emocje.....	5
Cele i zadania	6
Cele strategiczne	6
Zadania realizacyjne	6
Architektura systemu	7
Architektura ogólna	7
Przepływ danych.....	8
Technologie.....	8
Backend.....	8
Frontend	9
Infrastruktura.....	9
Struktura projektu	10
Drzewo katalogów	10
Komponenty systemu	10
Frontend (React + TypeScript).....	10
Główne komponenty.....	10
Cechy	11
Architektura i struktura.....	12
Konfiguracja	13
Komponenty	13
Komponenty formularzy.....	15
Komponenty danych i wyników.....	16
Komponenty UI i nawigacji	17
Komponenty autentykacji: Auth/	18
API.....	19
Plik: components/api/api.ts	19
Autentykacja.....	20
Typy TypeScript	21
Interfejsy komponentów.....	21
Instalacja i uruchomienie	22
Instalacja zależności	22

Uruchomienie w trybie development.....	22
Build do produkcji.....	23
Preview built version	23
Stylowanie.....	23
Zmienne tematu: MainDashboard.tsx.....	23
Struktura CSS	23
System kolorów.....	24
LocalStorage	24
Przepływ danych	24
Zmienne środowiska	25
Notatki dla deweloperów	25
Ścieżka weryfikacji	25
Backend API (FastAPI).....	26
Zakres i cel.....	26
Źródła danych i mapowanie emocji	27
Preprocessing danych (data_preprocessing.ipynb)	28
Trening modelu tekstowego (model_training.ipynb).....	28
Trening modelu obrazowego - wersja bazowa (backend/model_training.ipynb)	29
Trening modelu obrazowego - wersja rozszerzona (backend/models/model_training.ipynb)	29
Analiza zbiorów (moodify_datasets_analysis.ipynb).....	30
Test predykcji (backend/models/test_predictions.ipynb).....	30
Keras Tuner (backend/models/keras_tuner).....	30
Zależności.....	30
Typowy przebieg pracy.....	31
Uwaga o duplikacji	31
Firestore.....	31
Informacje ogólne.....	31
Konfiguracja Klienta (Frontend)	31
Zainicjalizowane usługi.....	31
Obiekt konfiguracyjny	31
Konfiguracja Serwera (Backend)	32
Backend API (api/services/firebase.py)	32
Frontend.....	33
Baza Danych (Cloud Firestore)	33
Struktura Kolekcji	33

Operacje na Danych (Serwisy)	34
Endpointy API powiązane z Firebase	34
Machine Learning	34
Datasety	34
AffectNet	34
Emotion Dataset.....	34
Modele.....	35
Model CNN dla obrazów.....	35
Model SVM dla tekstu	35
Metryki	36
Preprocessing	36
Instrukcje instalacji	36
Wymagania systemowe	36
Konfiguracja wstępna.....	37
Clone repozytorium	37
Przygotowanie Firebase	37
Spotify API	37
Backend Setup	37
Frontend Setup	38
Weryfikacja instalacji.....	38
Backend	38
Frontend.....	38
Spotify	38
Wnioski	39
Podsumowanie.....	39
Osiągnięcia	39
Mocne strony	39
Obszary do usprawnienia.....	39
Rekomendacje.....	40
Potencjał biznesowy	40
Kontakt i wsparcie	40

Streszczenie wykonawcze

Moodify to zaawansowana aplikacja webowa łącząca sztuczną inteligencję z muzyką. System analizuje emocje użytkownika z dwóch źródeł (tekst i obraz) za pomocą modeli Machine Learning, a następnie automatycznie dobiera muzykę ze Spotify pasującą do bieżącego nastroju użytkownika.

Kluczowe cechy:

- Rozpoznawanie emocji z tekstu i obrazu (AI/ML)
- Integracja ze Spotify API
- Autentykacja przez Firebase
- Przechowywanie historii nastrojów
- Responsywny interfejs webowy
- Tryb ciemny/jasny

Opis projektu

Cel główny

Stworzenie inteligentnego systemu, który:

1. **Rozpoznaje emocje** użytkownika z tekstu i obrazu za pomocą modeli ML
2. **Mapuje emocje** na kategorie muzyczne
3. **Dobiera playlisty** ze Spotify odpowiadające nastrojowi
4. **Personalizuje doświadczenie** na podstawie historii użytkownika

Rozpoznawane emocje

5 wspólnych emocji (tekst + obraz):

- **Szczęśliwy/Radość** - muzyka energetyczna, wesola
- **Smutny/Smutek** - muzyka spokojna, refleksyjna
- **Energiczny/Gniew** - muzyka intensywna, dynamiczna
- **Spokojny/Strach** - muzyka uspokajająca, ambient
- **Zaskoczony/Zaskoczenie** - muzyka zróżnicowana, ekscytująca

Dodatkowa emocja dla obrazów:

- **Neutralny** - muzyka background, uniwersalna

Cele i zadania

Cele strategiczne

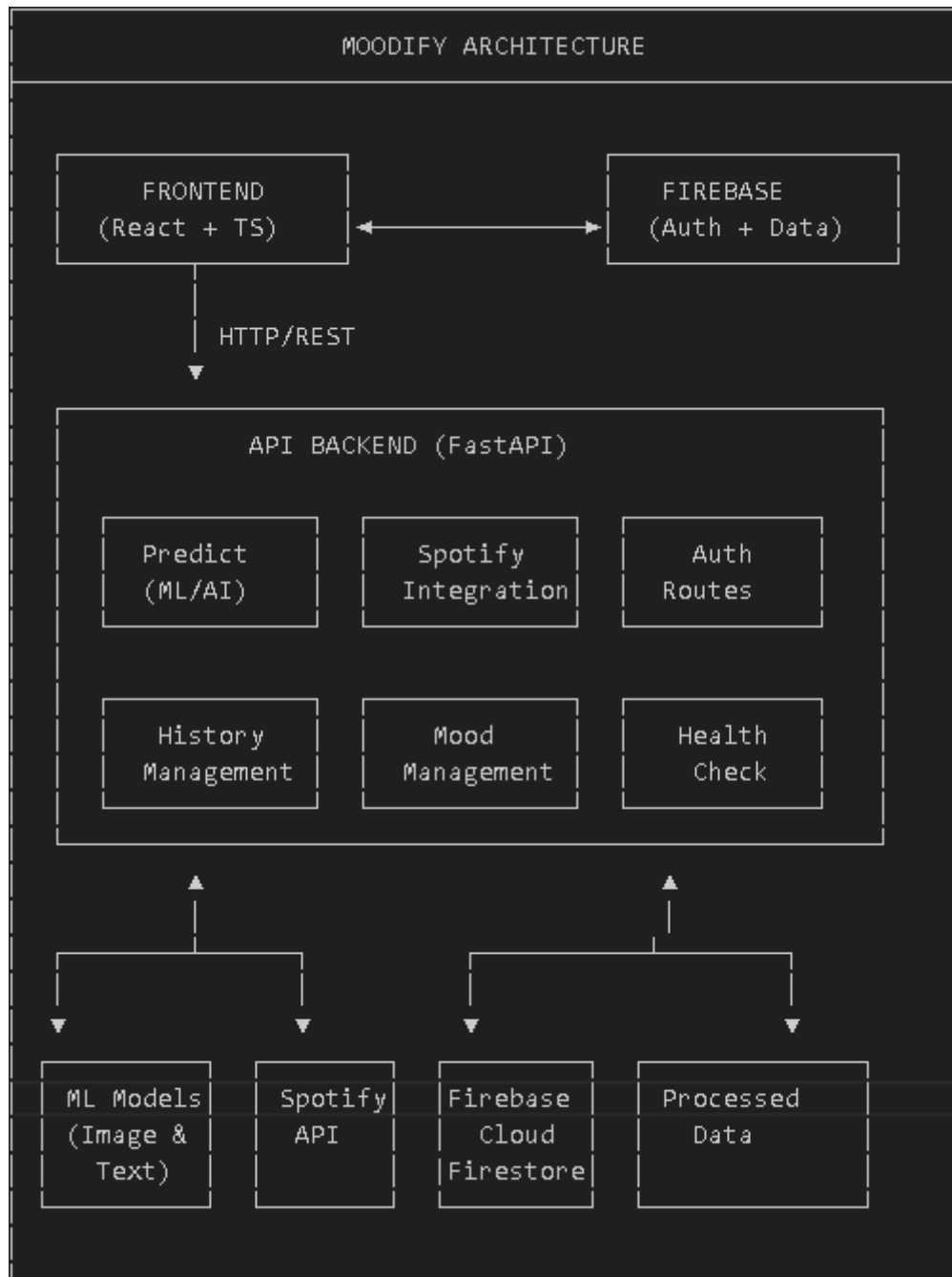
- Implementacja systemów AI/ML do rozpoznawania emocji
- Integracja z Spotify API
- Stworzenie nowoczesnego interfejsu użytkownika
- Zapewnienie bezpiecznej autentykacji

Zadania realizacyjne

- Pobieranie i przetwarzanie datasetów (AffectNet, Emotion Dataset)
- Trenowanie modeli dla tekstu (SVM/BERT)
- Trenowanie modeli dla obrazów (CNN/Keras)
- Implementacja API (FastAPI)
- Budowa frontendu (React + TypeScript)
- Konfiguracja Firebase

Architektura systemu

Architektura ogólna



Przepływ danych

1. **Użytkownik** → **Frontend** - Logowanie, input tekstu/zdjęcia
2. **Frontend** → **API** - Wysłanie danych do analizy
3. **API** → **ML Models** - Analiza emocji
4. **ML Models** → **API** - Zwrócenie predicted emotion
5. **API** → **Spotify API** - Pobranie playlist
6. **API** → **Firebase** - Zapisanie historii
7. **API** → **Frontend** - Zwrócenie playlist i wyniku
8. **Frontend** → **User** - Wyświetlenie rekomendacji

Technologie

Backend

Kategoria	Technologia	Wersja	Cel
Framework	FastAPI	–	REST API
ML / AI	TensorFlow	–	Modele CNN
ML / AI	Keras	–	Definiowanie architektur
ML / AI	Scikit-learn	–	SVM dla tekstu
Python	Python	3.10+	Język podstawowy
Baza danych	Firebase Firestore	–	Przechowywanie danych
Autoryzacja	Firebase Auth	–	Autentykacja użytkowników
Storage	Firebase Storage	–	Przechowywanie zdjęć
Integracja	Spotify API	–	Pobieranie piosenek
Utilities	Python-dotenv	–	Zmienne środowiska
Data	NumPy	–	Operacje numeryczne
Data	Pandas	–	Przetwarzanie danych
Image	Pillow	–	Przetwarzanie obrazów

Frontend

Kategoria	Technologia	Wersja	Cel
Framework	React	18.3.1	Biblioteka UI
Język	TypeScript	5.6.2	Typowanie statyczne
Bundler	Vite	5.4.1	Szybki build tool
CSS	Tailwind CSS	3.4.13	Utility-first styling
Autentykacja	Firebase SDK	12.6.0	Auth kliencka
HTTP	Axios	1.13.2	HTTP client
Ikony	Lucide React	0.553.0	Icon library
Inne	React-DOM	18.3.1	Rendering

Infrastruktura

- **Backend Hosting** - Python/FastAPI (localhost:8000)
- **Frontend Hosting** - Vite dev server (localhost:5173)
- **Baza danych** - Firebase Firestore
- **Storage** - Firebase Storage
- **Autentykacja** - Firebase Authentication
- **API Externy** - Spotify Web API

Struktura projektu

Drzewo katalogów

```
moodify-main/
|
├── api/ (FastAPI)
|   ├── routes/      # Endpoints REST
|   ├── services/    # Logika biznesowa
|   └── data/        # Dane testowe
|
├── backend/ (ML)
|   ├── models/
|   |   ├── processed_data/
|   |   ├── saved_models/
|   |   └── notebooks/
|   └── README.md
|
├── frontend/ (React)
|   ├── components/
|   ├── api/
|   └── firebase/
|
└── docs/
```

Komponenty systemu

Frontend (React + TypeScript)

Główne komponenty

App.tsx - Zarządzanie autentykacją, routing

MainDashboard.tsx - Główny interfejs (443 linie)

State management: nastrój, zdjęcie, playlisty, etc.

Integracja wszystkich modułów

Obsługa motywu (dark/light)

Moduły funkcjonalne:

- **Auth** - Logowanie, rejestracja, profil
- **CameraBox** - Upload i podgląd zdjęć
- **AddMoodForm** - Formularz opisanego nastroju
- **MoodSelectPanel** - Wybór z dostępnych nastrojów
- **SongsList/PlaylistsList** - Wyświetlanie wyników
- **Recommendations** - Podsumowanie rekomendacji
- **AnalysisResult** - Wynik analizy emocji

Cechy

Responsywny design (Tailwind CSS)

Dark/Light mode

Real-time aktualizacja

Hot reload (Vite HMR)

Stack technologiczny:

- React 18.3.1
- TypeScript 5.6.2
- Vite 5.4.1 (bundler)
- Tailwind CSS 3.4.13
- Firebase 12.6.0
- Axios 1.13.2
- Lucide React 0.553.0 (ikony)

Architektura i struktura

Struktura katalogów

```
frontend/
├── src/
│   ├── components/                # Komponenty React
│   │   ├── Auth/                  # Komponenty autentykacji
│   │   │   ├── Auth.css
│   │   │   ├── AuthPopup.tsx
│   │   │   ├── LoginForm.tsx
│   │   │   ├── RegisterForm.tsx
│   │   │   ├── ProfileModal.tsx
│   │   │   └── UserPanel.tsx
│   │   ├── api/                   # Integracja z API
│   │   │   └── api.ts
│   │   ├── firebase/              # Konfiguracja Firebase
│   │   │   └── firebase.ts
│   │   ├── types/                 # Typy TypeScript
│   │   │   └── types.ts
│   │   ├── assets/                # Zasoby statyczne (logo, obrazy)
│   │   ├── AddMoodForm.tsx        # Formularz dodawania nastroju
│   │   ├── AnalysisResult.tsx     # Wynik analizy emocji
│   │   ├── CameraBox.tsx          # Interfejs kamery
│   │   ├── Card.tsx               # Komponenta karty
│   │   ├── ControlToggles.tsx     # Przyciski przełączników
│   │   ├── DescriptionBox.tsx     # Pole opisu
│   │   ├── GenerateButton.tsx     # Przycisk generowania
│   │   ├── HistoryList.tsx        # Lista historii
│   │   ├── LoginPanel.tsx         # Panel logowania
│   │   ├── MainDashboard.tsx      # Główny komponent dashboardu
│   │   ├── MoodSelectPanel.tsx    # Panel wyboru nastroju
│   │   ├── MoodsList.tsx          # Lista nastrojów
│   │   ├── Recommendations.tsx    # Rekomendacje
│   │   ├── SongsList.tsx          # Lista piosenek
│   │   ├── ThemeToggle.tsx        # Przełącznik motywu
│   │   ├── UserPanel.tsx          # Panel użytkownika
│   │   └── vite-end.d.ts          # Deklaracje typów Vite
│   ├── App.tsx                    # Główny komponent (routing)
│   ├── main.tsx                   # Punkt wejścia
│   └── index.css                   # Style globalne
├── public/                        # Zasoby publiczne
├── vite.config.ts                  # Konfiguracja Vite
├── tsconfig.json                   # Konfiguracja TypeScript
├── tailwind.config.js              # Konfiguracja Tailwind CSS
├── postcss.config.js               # Konfiguracja PostCSS
└── package.json                    # Zależności projektu
```

Konfiguracja

Vite Configuration (vite.config.ts)

typescript

port: 5173

host: true (dostępne z zewnątrz)

proxy API: "/api" → "http://localhost:8000"

TypeScript (tsconfig.json)

Target: ES2020

Module: ESNext

JSX: react-jsx

Strict mode: enabled

Module resolution: Bundler

Tailwind CSS

Stylowanie oparte na utility-first framework:

- Niestandardowe gradienty dla ciemnego i jasnego motywu
- Responsywny design
- Dark mode support
- Autoprefixer dla kompatybilności przeglądarek

Komponenty

Główny punkt wejścia: App.tsx

Funkcjonalność:

- Zarządzanie stanem autentykacji
- Wyświetlanie loading state
- Warunkowe renderowanie: MainDashboard (zalogowany) lub AuthPopup (niezalogowany)
- Monitoruje onAuthStateChanged z Firebase
- Wyświetla loading screen do czasu załadowania stanu
- Renderuje odpowiedni komponenty na podstawie stanu autentykacji

Stan:

typescript

```
const [user, setUser] = useState<User | null>(null)
```

```
const [loading, setLoading] = useState(true)
```

Dashboard: MainDashboard.tsx

Główny komponent aplikacji - centralna część interfejsu (443 linie)**State:**

Pole	Typ	Opis
theme	"dark" "light"	Aktualny motyw aplikacji
selectedModule	"description" "camera" "mood" null	Aktualnie wybrany moduł
description	string	Opis nastroju podany przez użytkownika
selectedMood	MoodType	Nastrój wybrany ręcznie (enum)
analysisMood	MoodType null	Nastrój wykryty przez analizę AI
analysisSourceLabel	string null	Źródło analizy nastroju
tracks	TrackItem []	Lista utworów ze Spotify
playlists	PlaylistItem []	Lista playlist
photoPreview	string null	Podgląd zdjęcia (base64)
photoFileName	string null	Nazwa pliku zdjęcia
photoFile	File null	Obiekt pliku zdjęcia
detectedEmotion	string null	Emocja wykryta przez model
isGenerating	boolean	Czy trwa generowanie danych
generatedAt	string null	Timestamp wygenerowania

Główne funkcje:

- handlePhotoUpload(file) - przesyła zdjęcie i wysyła do API
- getTracksfromDB() - pobiera piosenki z historii backendu
- toggleTheme() - przełącza między ciemnym a jasnym motywem
- Zarządzanie stanem UI dla modułów (kamera, opis, nastrój)

Stałe:

typescript

```
gradient = "bg-[radial-gradient(...)]" // Gradient ciemny
```

```
lightGradient = "bg-[radial-gradient(...)]" // Gradient jasny
```

```
MODULE_LABELS = { description, camera, mood } // Etykiety
```

```
THEME_STORAGE_KEY = "moodify-theme" // Klucz localStorage
```

Komponenty formularzy

AddMoodForm.tsx

Formularz do dodawania nowego nastroju:

- Pole na opis
- Selektor daty
- Przycisk wysyłki

MoodSelectPanel.tsx

Panel do wyboru typu nastroju

Dostępne nastroje:

- Szczęśliwy
- Smutny
- Spokojny
- Energiczny
- Zaskoczony
- Wizualny interfejs wyboru

DescriptionBox.tsx

Pole tekstowe do wpisania opisu nastroju:

- Textarea z placeholder
- Walidacja

CameraBox.tsx

Interfejs do:

- Wgrywania zdjęcia z urządzenia (file input)
- Podglądu zdjęcia przed wysyłką
- Wysyłki do API do analizy emocji (computer vision)

Komponenty danych i wyników

SongsList.tsx

Wyświetla listę piosenek ze Spotify

typescript

```
export interface TrackItem {  
  
  id: string  
  
  name: string  
  
  artists: Artist[]  
  
  album: Album  
  
  preview_url?: string  
  
}
```

PlaylistsList.tsx

Wyświetla listę list odtwarzania

typescript

```
export interface PlaylistItem {  
  
  id: string  
  
  name: string  
  
  description?: string  
  
  images: Image[ ]  
  
  external_urls: ExternalUrls  
  
}
```


MoodsList.tsx

Wyświetla historię nastrojów użytkownika

- Pobiera dane z API /api/history/items/
- Pozwala na usunięcie wpisów
- Odświeża się automatycznie
- Wyświetla datę i godzinę w formacie PL

Recommendations.tsx / RecommendationsSummary.tsx

Wyświetla streszczenie rekomendacji muzycznych

- Na podstawie analizy nastroju
- Ilość piosenek
- Ilość playlist

AnalysisResult.tsx

Pokazuje rezultaty analizy zdjęcia:

- Wykryta emocja
- Poziom pewności/confidence
- Rekomendacje na podstawie emocji

Komponenty UI i nawigacji

Card.tsx

Komponent karty - uniwersalny element UI do grupowania zawartości

- Wrapper z stylami
- Obsługuje children

ControlToggles.tsx

Przyciski do przełączania między modułami

typescript

```
type ToggleKey = "description" | "camera" | "mood" | null
```

- 3 główne moduły
- Aktualny stan wyróżniony

GenerateButton.tsx

Przycisk do uruchomienia generowania rekomendacji

- Wysyła żądanie do API
- Pokazuje loading state
- Obsługuje błędy

ThemeToggle.tsx

Przycisk do przełączania między motywem jasnym a ciemnym

typescript

```
type Theme = "dark" | "light"

interface ThemeToggleProps {

  theme: Theme

  onToggle: () => void

}
```

Komponenty autentykacji: Auth/

AuthPopup.tsx

Modal wyświetlany niezalogowanym użytkownikom

- Zawiera formularze logowania i rejestracji
- Przełączanie między formami

LoginForm.tsx

Formularz logowania:

- Email i hasło
- Walidacja
- Przycisk logowania
- Link do rejestracji

RegisterForm.tsx

Formularz rejestracji

- Email, hasło, potwierdzenie hasła
- Walidacja (hasła muszą się zgadzać)
- Przycisk rejestracji

ProfileModal.tsx

Modal profilu zalogowanego użytkownika

- Dane profilu (email)
- Opcje wylogowania
- Zamykanie modalu

UserPanel.tsx

Panel użytkownika w dashboardzie

typescript

```
interface UserPanelProps {  
    user: User | null  
}
```

- Pokazuje dane bieżącego użytkownika
- Przycisk otwierający profil
- Przycisk wylogowania

API

Plik: `components/api/api.ts`

Zawiera funkcje do komunikacji z backendem:

Mock API (do testowania bez backendu)

typescript

```
mockAddMoodApi(mood: MoodType, description: string, date: string)  
    → Promise<{ status: 'success' | 'error', data?: Mood, message?: string }>  
  
mockGetMoodsApi()  
    → Promise<Mood[ ]>  
  
mockDeleteMoodApi(id: string)  
    → Promise<{ status: 'success' | 'error' }>
```

API Backend (rzeczywiste)

Żądania HTTP do:

- POST /api/history/items/ - pobieranie historii nastrojów
- POST /api/predict - wysyłanie zdjęcia do analizy emocji
- GET /api/spotify/ - integracja ze Spotify
- POST /api/mood/ - zarządzanie nastrojami (create, delete)
- GET /api/health - sprawdzenie zdrowotności API

Proxy konfiguracja (Vite):

/api → http://localhost:8000

Autentykacja

Konfiguracja Firebase: components/firebase/firebase.ts

Obsługiwane metody:

- Email/hasło (primaryEmail authentication)
- Potencjalnie OAuth (Google, Facebook - do implementacji)

Funkcjonalność:

- Tworzenie nowych kont (createUserWithEmailAndPassword)
- Logowanie (signInWithEmailAndPassword)
- Wylogowanie (signOut)
- Resetowanie hasła
- Pobieranie ID tokena do żądań API
- Pobieranie danych użytkownika (auth.currentUser)

Integracja:

- App.tsx monitoruje zmiany stanu autentykacji przez onAuthStateChanged
- Komponenty Auth obsługują formularze
- UserPanel wyświetla dane zalogowanego użytkownika
- Token pobierany w żądaniach API: auth.currentUser.getIdToken()

Typy TypeScript

components/types/types.ts

typescript

```
export interface Mood {  
  
  id: string  
  
  mood: MoodType      // Typ nastroju  
  
  description: string  // Opis nastroju  
  
  date: string         // Data  
  
  timestamp: number    // Unix timestamp  
  
}  
  
export type MoodType =  
  
  | 'Szczęśliwy'  
  
  | 'Smutny'  
  
  | 'Spokojny'  
  
  | 'Energiczny'  
  
  | 'Zaskoczony'
```

Interfejsy komponentów

typescript

```
interface TrackItem {  
  
  id: string  
  
  name: string  
  
  artists: Artist[]  
  
  album: Album  
  
}  
  
interface PlaylistItem {
```

```
    id: string

    name: string

    description?: string

    images: Image[]
  }

  interface UserPanelProps {

    user: User | null

  }

  interface ThemeToggleProps {

    theme: "dark" | "light"

    onToggle: () => void

  }
```

Instalacja i uruchomienie

Wymagania

- Node.js 18+
- npm lub yarn
- Backend API uruchomiony na <http://localhost:8000>

Instalacja zależności

```
bash
```

```
cd frontend
```

```
npm install
```

Uruchomienie w trybie development

```
bash
```

```
npm run dev
```

- Aplikacja będzie dostępna na: <http://localhost:5173>
- Hot reload włączony
- Proxy do API skonfigurowany

Build do produkcji

bash

```
npm run build
```

- Skompiluje TypeScript (tsc -b)
- Zoptymalizuje z Vite
- Wyjście w katalogu dist/
- Gotowe do deploymentu

Preview built version

bash

```
npm run preview
```

- Uruchamia built version lokalnie

Stylowanie

Framework: Tailwind CSS

Główne cechy:

- Utility-first CSS framework
- Responsive design (mobile-first)
- Dark mode support
- PostCSS dla autoprefixowania

Zmienne tematu: MainDashboard.tsx

typescript

```
const gradient = "bg-[radial-gradient(1300px_700px_at_5%_35%,...)]"
```

```
const lightGradient = "bg-[radial-gradient(1200px_600px_at_80%_-10%,...)]"
```

Gradienty definiują tło aplikacji dla różnych motywów.

Struktura CSS

- index.css - style globalne i setup Tailwind
- Auth/Auth.css - style dla komponentów autentykacji
- Komponenty mają inline Tailwind classes
- Brak oddzielnych SCSS/LESS plików

System kolorów

Ciemny motyw (domyślny):

- Tła: czarne i ciemnoszare (05090f, 020308)
- Akcenty: zielone (emerald) i fioletowe (indigo)
- Tekst: biały

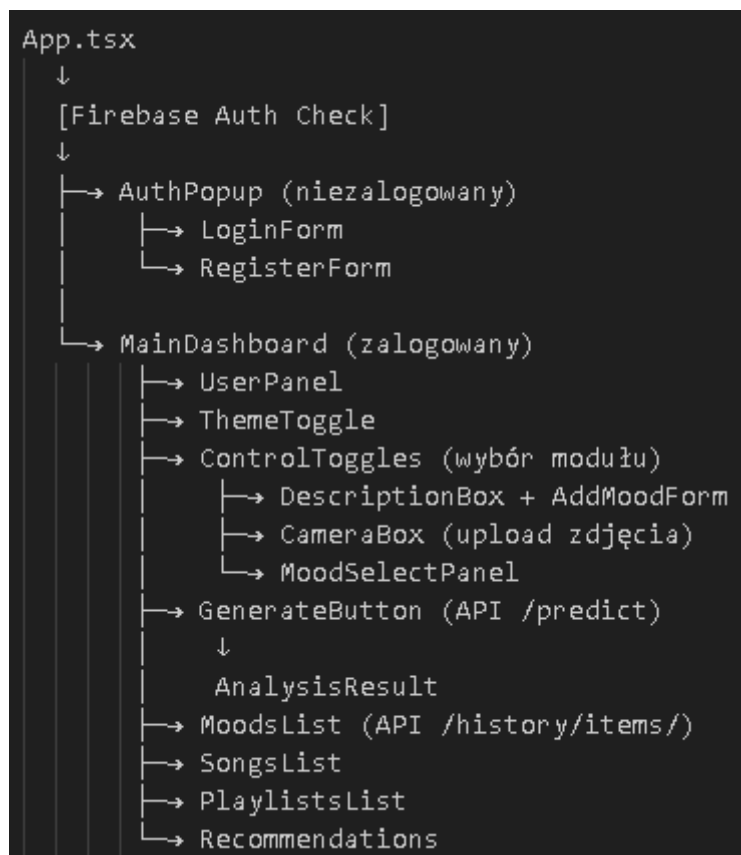
Jasny motyw:

- Tła: białe i jasne szare (rgba(148,163,184,0.35), white)
- Akcenty: fioletowe i zielone
- Tekst: ciemny szary

LocalStorage

- Motyw zapisywany w localStorage z kluczem moodify-theme
- Persystuje na pomiędzy sesjami

Przepływ danych



Zmienne środowiska

Konfiguracja Firebase powinna być w components/firebase/firebase.ts

Wymagane stałe Firebase:

typescript

```
const firebaseConfig = {  
  apiKey: "YOUR_API_KEY",  
  authDomain: "YOUR_AUTH_DOMAIN",  
  projectId: "YOUR_PROJECT_ID",  
  storageBucket: "YOUR_STORAGE_BUCKET",  
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",  
  appId: "YOUR_APP_ID"  
}
```

Notatki dla deweloperów

1. **TypeScript Strict Mode** - wszystkie typy muszą być wyraźnie zdefiniowane
2. **React Hooks** - komponenty używają useState, useEffect, nie ma komponentów klasowych
3. **Firebase Auth** - bieżący użytkownik dostępny przez auth.currentUser
4. **API Proxy** - żądania do /api/ są automatycznie routowane do backendu (Vite)
5. **Theme Persistence** - motyw zapisywany w localStorage z kluczem moodify-theme
6. **Hot Reload** - Vite zapewnia HMR dla szybkiego developmentu
7. **CORS** - backend musi pozwolić na żądania z localhost:5173

Ścieżka weryfikacji

Aby sprawdzić czy aplikacja działa:

1. Uruchom backend: python api/main.py
2. Uruchom frontend: npm run dev
3. Przejdź do http://localhost:5173
4. Zaloguj się lub utwórz konto
5. Spróbuj:
 - Dodać nastrój przez opis
 - Wgrać zdjęcie do analizy

- Wybrać nastrój z panelu
- Generować rekomendacje
- Przełączyć motyw

Backend API (FastAPI)

Zakres i cel

Backend zawiera notebooki i artefakty do:

- pobierania i analizy zbiorów danych (AffectNet, Emotion)
- preprocessingu danych tekstowych i obrazowych
- trenowania modeli tekstowych i obrazowych
- strojenia hiperparametrow oraz ewaluacji

Struktura folderu

Ponizej opis kluczowych plików i katalogów. W backend są dwie warstwy:

- backend/ - wersja bazowa pipeline'u
- backend/models/ - wersja rozszerzona (tuning, dodatkowe notebooki i logi)

Struktura katalogu backend

Ścieżka	Typ	Opis
backend/data_preprocessing.ipynb	Notebook	Pobieranie danych oraz preprocessing
backend/model_training.ipynb	Notebook	Trening modeli – wersja bazowa
backend/moodify_datasets_analysis.ipynb	Notebook	Analiza i wizualizacja zbiorów danych
backend/processed_data/	Katalog	Przetworzone dane (CSV) i konfiguracja ścieżek
backend/saved_models/	Katalog	Wytrenowane modele oraz mapy etykiet

Struktura katalogu backend/models

Ścieżka	Typ	Opis
backend/models/data_preprocessing.ipynb	Notebook	Pobieranie danych i preprocessing (druga kopia)
backend/models/model_training.ipynb	Notebook	Trening modeli + augmentacje + Keras Tuner
backend/models/moodify_datasets_analysis.ipynb	Notebook	Analiza zbiorów danych (druga kopia)
backend/models/tensorboard_viewer.ipynb	Notebook	Podgląd i analiza logów TensorBoard
backend/models/test_predictions.ipynb	Notebook	Test predykcji na zbiorach obrazów
backend/models/processed_data/	Katalog	CSV-y i konfiguracja ścieżek (druga kopia)
backend/models/saved_models/	Katalog	Wytrenowane modele (druga kopia)
backend/models/keras_tuner/	Katalog	Wyniki strojenia Hyperband + logi

Źródła danych i mapowanie emocji

Notebook data_preprocessing.ipynb korzysta z:

- AffectNet przez kagglehub.dataset_download("mstjebashazida/affectnet")
- Emotion dataset przez load_dataset("dair-ai/emotion")

Mapowanie etykiet dla Emotion (HF):

- 0 Sadness -> Smutek
- 1 Joy -> Szczescie
- 2 Love -> Szczescie
- 3 Anger -> Zlosc
- 4 Fear -> Strach
- 5 Surprise -> Zaskoczenie

Mapowanie folderow AffectNet:

- happy -> Szczescie
- sad -> Smutek
- anger -> Zlosc
- fear -> Strach
- surprise -> Zaskoczenie

Finalnie uzywane jest 5 klas: Szczescie, Smutek, Zlosc, Strach, Zaskoczenie.

Preprocessing danych (data_preprocessing.ipynb)

Funkcje i efekty:

- pobranie AffectNet i Emotion
- normalizacja etykiet do 5 wspólnych emocji
- przygotowanie CSV dla tekstu i obrazów
- zapis konfiguracji data_paths.json z bazową ścieżką do AffectNet

Podział danych obrazowych:

- 70% train, 15% val, 15% test
- stratyfikacja po etykietach (train_test_split z stratify)

Formaty plików:

- processed_data/train_text.csv, val_text.csv, test_text.csv
- kolumny: text, emotion
- processed_data/train_images.csv, val_images.csv, test_images.csv
- kolumny: path, emotion
- path jest ścieżką względną do AffectNet
- processed_data/data_paths.json
- affectnet_base_path to baza do budowania pełnych ścieżek

Uwaga o data_paths.json:

- w backend/processed_data zapisano ścieżkę absolutną do cache KaggleHub
- w backend/models/processed_data zapisano ścieżkę względną (../data)

Trening modelu tekstowego (model_training.ipynb)

Algorytm i konfiguracja:

- wektoryzacja: TfidfVectorizer(max_features=5000, min_df=2, max_df=0.8, ngram_range=(1, 2))
- klasyfikator: SVC(kernel='linear')
- ewaluacja: accuracy, classification report, confusion matrix

Artefakty:

- saved_models/svm_text_model.pkl
- saved_models/tfidf_vectorizer.pkl

Trening modelu obrazowego - wersja bazowa (backend/model_training.ipynb)

Konfiguracja:

- baza: MobileNetV2 (ImageNet, include_top=False)
- wejscie: 224x224, batch 32
- head: GlobalAveragePooling2D -> Dense(128) -> Dropout(0.5) -> Dense(5, softmax)
- optymalizator: Adam, learning_rate=1e-4
- trening: do 20 epok, early stopping, checkpoint na val_accuracy

Artefakty:

- saved_models/image_model_best.h5
- saved_models/image_model_final.h5
- saved_models/image_emotion_map.json (mapa id -> emocja)

Trening modelu obrazowego - wersja rozszerzona (backend/models/model_training.ipynb)

Dodatki i roznice wzgledem wersji bazowej:

- fragment konfiguracji CUDA (LD_LIBRARY_PATH, ctypes.CDLL) dla TensorFlow
- augmentacje obrazu: flip, brightness, contrast, saturation, rotacje, zoom/crop
- filtracja wierszy z emotion == neutral (jesli wystapia)

Tuning Hyperband (Keras Tuner):

- max_epochs=10, factor=3, hyperband_iterations=1
- hiperparametry: dropout_1 (0.3/0.5), dropout_2 (0.3/0.4),
- learning_rate (0.0003/0.001), units_1 (256), units_2 (192)
- fine-tuning ostatnich 20 warstw MobileNetV2
- logi w models/keras_tuner/tensorboard_logs

Finalny trening:

- do 50 epok, EarlyStopping i ReduceLROnPlateau
- checkpoint saved_models/image_model_best.h5
- zapis finalny saved_models/image_model_final.h5

Analiza zbiorow (moodify_datasets_analysis.ipynb)

Notebook do sanity check:

- przegląd struktury AffectNet i Emotion
- losowe przykłady, rozkłady klas, statystyki
- mapowanie emocji do 5 wspólnych klas

Test predykcji (backend/models/test_predictions.ipynb)

Cel i dane:

- ładowanie saved_models/image_model_final.h5 i image_emotion_map.json
- ewaluacja na processed_data/test_images.csv
- dodatkowe testy na processed_data/data_test/ (lokalne obrazy)

Ścieżki danych:

- BASE_PATH pobierany z KaggleHub albo ustawiony na lokalną ścieżkę
- wymaga aktualnego data_paths.json lub poprawnego BASE_PATH

Keras Tuner (backend/models/keras_tuner)

Struktura:

- moodify_hyperband_fast/ i moodify_hyperband_finetuned/
- trial_XXXX/ z plikami trial.json, build_config.json, checkpoint.weights.h5
- tensorboard_logs/ - logi TensorBoard dla poszczególnych uruchomień

Notebook do podglądu:

- backend/models/tensorboard_viewer.ipynb
- komenda: %tensorboard --logdir keras_tuner/tensorboard_logs

Zależności

Najczęściej używane biblioteki:

- kagglehub, datasets (Hugging Face)
- pandas, numpy
- scikit-learn
- tensorflow, keras, keras-tuner
- matplotlib, seaborn

- Pillow (PIL), opencv-python (cv2)

Typowy przebieg pracy

1. Uruchom `data_preprocessing.ipynb` (generuje `processed_data/`).
2. Wybierz jedną wersję treningu:
 - `backend/model_training.ipynb` (bazowa)
 - `backend/models/model_training.ipynb` (rozszerzona + tuning)
3. (Opcjonalnie) `backend/models/tensorboard_viewer.ipynb` dla logów.
4. (Opcjonalnie) `backend/models/test_predictions.ipynb` dla ewaluacji.

Uwaga o duplikacji

Pliki w `backend/` i `backend/models/` mają podobne nazwy, ale nie są identyczne. Wersja w `backend/models/` zawiera tuning, augmentacje i dodatkowe notebooki, więc przy pracy w praktyce warto trzymać się jednej "warstwy" aby uniknąć rozjazdu artefaktów.

Firestore

Informacje ogólne

ID Projektu: moodify-1c59b

Domena Auth: moodify-1c59b.firebaseio.com

Bucket Storage: moodify-1c59b.appspot.com

Hosting: Skonfigurowany dla aplikacji typu SPA (Single Page Application)

Konfiguracja Klienta (Frontend)

Inicjalizacja po stronie klienta znajduje się w pliku

`frontend/src/components/firebase/firebase.ts`. Aplikacja wykorzystuje modułowe SDK

Firestore v9+.

Zainicjalizowane usługi

- **Authentication** (`getAuth`)
- **Firestore** (`getFirestore`)

Obiekt konfiguracyjny

Kod zawiera jawną konfigurację dla środowiska produkcyjnego/deweloperskiego:

- `authDomain: moodify-1c59b.firebaseio.com`

- projectId: moodify-1c59b
- storageBucket: moodify-1c59b.appspot.com
- messagingSenderId: 102165390463
- appId: 1:102165390463:web:02c5ffb694692230063b6c

Konfiguracja Serwera (Backend)

Backend napisany w Pythonie (FastAPI) wykorzystuje firebase-admin SDK do operacji uprzywilejowanych.

Inicjalizacja

Lokalizacja: api/main.py

Serwer szuka pliku klucza konta serwisowego service-account.json w głównym katalogu. Jeśli plik istnieje, aplikacja jest inicjalizowana z uprawnieniami administratora i dostępem do Storage Bucket.

Zmienne środowiskowe

Wymagane zmienne w pliku .env dla obsługi Firebase:

FIREBASE_API_KEY: Klucz Web API wymagany do logowania użytkowników przez REST API.

FIREBASE_STORAGE_BUCKET: (Opcjonalnie) Adres bucketu, domyślnie moodify-1c59b.appspot.com.

Uwierzytelnianie

System obsługuje uwierzytelnianie hybrydowe, łącząc SDK klienta z REST API po stronie serwera.

Metody logowania

Email/Hasło: Główna metoda uwierzytelniania.

Backend API (api/services/firebase.py)

Backend udostępnia funkcje pomocnicze:

create_user_in_firebase: Tworzy użytkownika przy użyciu auth.create_user (Admin SDK). Ustawia email, password oraz display_name.

log_into_firebase: Wykonuje żądanie HTTP POST do `identitytoolkit.googleapis.com` w celu zalogowania użytkownika hasłem i uzyskania tokena (ponieważ Admin SDK nie służy do logowania użytkowników hasłami).

Frontend

Frontend nasłuchuje zmian stanu uwierzytelnienia za pomocą `onAuthStateChanged` w pliku `App.tsx`, co pozwala na warunkowe renderowanie widoków (`MainDashboard` vs `AuthPopup`).

Baza Danych (Cloud Firestore)

Projekt wykorzystuje bazę NoSQL Firestore do przechowywania historii nastrojów użytkowników.

Struktura Kolekcji

Kolekcja users

Główna kolekcja przechowująca dokumenty użytkowników.

ID Dokumentu: UID użytkownika (z Firebase Auth)

Pod kolekcją moods

Ścieżka: `users/{uid}/moods`

Przechowuje historię wygenerowanych nastrojów i przypisanych piosenek.

Schemat dokumentu w moods:

Pole	Typ	Opis
artist	string	Wykonawca rekomendowanej piosenki
date	string	Data generowania w ustalonym formacie
desc	string null	Opis / dodatkowa informacja (opcjonalna)
duration_ms	int	Długość piosenki w milisekundach
generated_at	date	Timestamp wygenerowany po stronie Firestore
mood	string	Przewidziany nastrój
name	string	Tytuł piosenki
source	string	Moduł, z którego korzystał użytkownik
spotify_id	string	Identyfikator utworu w Spotify
spotify_url	string	Bezpośredni link do utworu w Spotify

Operacje na Danych (Serwisy)

Backend (`api/services/firebase.py`) implementuje następujące operacje na bazie danych:

save_mood_to_firestore(uid, mood_data): Dodaje nowy dokument do kolekcji moods danego użytkownika.

get_tracks_history_from_firestore(uid): Pobiera wszystkie dokumenty z podkolekcji moods dla danego użytkownika i zwraca je w formie słownika, gdzie kluczem jest ID dokumentu, a wartością dane utworu.

Endpointy API powiązane z Firebase

Zgodnie z dokumentacją API (`frontend/api_docs.md`), następujące endpointy interaktywują z Firebase:

POST /api/firebase/login/: Logowanie użytkownika (zwraca UID i token).

POST /api/recommendations/songs/: Zapisuje piosenkę i wszystkie informacje do Firestore, jeśli podano UID.

Machine Learning

Datasetsy

AffectNet

Zbiór: ~450,000 zdjęć twarzy

Emocje: 8 (neutral, happy, sad, surprise, fear, disgust, anger, contempt)

Użycie: Trenowanie modelu dla obrazów

Źródło: Kaggle, baza publiczna

Emotion Dataset

Zbiór: 20,000 tekstów

Emocje: 6 (sadness, joy, love, anger, fear, surprise)

Użycie: Trenowanie modelu dla tekstu

Źródło: Hugging Face, open-source

Modele

Model CNN dla obrazów

```
Input: 224x224 RGB image
|
↓
Conv2D(32 filters) + ReLU + MaxPool
|
↓
Conv2D(64 filters) + ReLU + MaxPool
|
↓
Flatten()
|
↓
Dense(128) + ReLU + Dropout(0.5)
|
↓
Output: Dense(6) + Softmax
|
↓
6 emocji (klasy)
```

Hiperparametry:

- Optimizer: Adam (lr=0.001)
- Loss: categorical_crossentropy
- Batch size: 32
- Epochs: 100 (early stopping)
- Validation split: 0.2

Model SVM dla tekstu

```
Input: Tekst
|
↓
TF-IDF Vectorizer
|
↓
SVM Classifier
|
↓
Output: 6 emocji
```

Hiperparametry:

- Kernel: rbf
- C: 1.0
- Gamma: auto
- Max features (TF-IDF): 5000

Metryki

Model	Typ danych	Zbiór	Accuracy	Precision	Recall	F1-Score
CNN	Obrazy	Test	~82%	–	–	–
SVM	Tekst	Test	~78%	–	–	–

Preprocessing

Dla obrazów:

- Resize do 224x224
- Normalizacja (0-1)
- Augmentacja: rotacja, flip, zoom
- Konwersja do RGB (jeśli gray)

Dla tekstu:

- Lowercase
- Usunięcie punctuation
- Tokenizacja
- TF-IDF wektoryzacja

Instrukcje instalacji

Wymagania systemowe

Python: 3.10+

Node.js: 18+

npm lub **yarn**

Git

(Opcjonalnie) CUDA/cuDNN dla GPU

Konfiguracja wstępna

Clone repozytorium

```
bash
```

```
git clone <repo-url>
```

```
cd moodify
```

Przygotowanie Firebase

Utwórz Firebase project na console.firebase.google.com

Pobierz `service-account.json`

Umieść w katalogu `api/` (`gitignore`)

Spotify API

Wejdź na developer.spotify.com

Utwórz aplikację

Skopiuj Client ID i Secret

Umieść w `api/.env`

Backend Setup

```
bash
```

```
cd api
```

Utwórz virtual environment:

```
python -m venv venv
```

Aktywuj (Windows):

```
.\venv\Scripts\Activate.ps1
```

Aktywuj (macOS/Linux):

```
source venv/bin/activate
```

Zainstaluj zależności:

```
pip install -r requirements.txt
```

Uruchom serwer:

```
uvicorn main:app --reload --port 8000
```

API dostępny: <http://localhost:8000>

Frontend Setup

```
bash
```

```
cd frontend
```

Zainstaluj zależności:

```
npm install
```

Uruchom dev server:

```
npm run dev
```

Lub build do produkcji:

```
npm run build
```

Frontend dostępny: <http://localhost:5173>

Weryfikacja instalacji

Backend

```
bash
```

```
curl http://localhost:8000/api/health
```

Powinno zwrócić: {"status": "ok"}

Frontend

Otwórz <http://localhost:5173>

Powinieneś zobaczyć login screen

Spotify

```
bash
```

```
curl http://localhost:8000/api/spotify/health
```

Powinna się zwrócić informacja o tokenie

Wnioski

Podsumowanie

Moodify to zaawansowany projekt łączący:

- Machine Learning (CNN + SVM)
- API Integration (Spotify)
- Modern Authentication (Firebase)
- Modern Web Stack (React + TypeScript)

Osiągnięcia

1. Sukcesie zintegrowali AI/ML z aplikacją webową
2. Implementacja dwóch kanałów analizy (tekst + obraz)
3. Personalizowana rekomendacja muzyki
4. Nowoczesny, responsywny interfejs
5. Bezpieczna autentykacja

Mocne strony

1. Unikalne połączenie AI + muzyki
2. Użyteczna funkcjonalność
3. Czysty, zorganizowany kod
4. Kompleksowa dokumentacja
5. Skalowalna architektura

Obszary do usprawnienia

1. Dokładność ML modeli (~80%) - potencjał do poprawy
2. Rate limiting dla API
3. Cache strategy dla Spotify
4. Error handling & validating
5. Unit/Integration tests
6. Monitoring & logging

Rekomendacje

Krótkoterminowe:

- Improve ML accuracy (więcej danych)
- Add unit tests
- Implement caching

Długoterminowe:

- Mobile app
- Social features
- Advanced analytics
- Multi-language support
- Better accessibility

Potencjał biznesowy

1. Rynek music recommendation - rosnący
2. Monetyzacja poprzez premium features
3. Expansja na mobile
4. Globalna dostępność
5. Partnership z platformami muzycznymi

Kontakt i wsparcie

Projekt: Moodify - Inteligentny System Rekomendacji Muzyki

Data: Styczeń 2026

Stack: Python + TypeScript + Firebase

Status: W aktywnym developmencie